Smart AutoML Pro Dashboard: An Interpretable Visual System for Automated Model Selection, Evaluation, and Prediction

Hassnain Khan
Department of Computer Engineering
Instructor: Dr Kamran Saeed
Bahria University Islamabad H-11
Email: hassnain32222000@gmail.com

***Abstract*—Automated Machine Learning (AutoML) has become an important approach for reducing manual effort in machine learning pipeline development. However, many existing AutoML frameworks are designed for expert users and performance optimization, which makes them less transparent for beginners. This paper presents *Smart AutoML Pro Dashboard*, an interpretable and visual AutoML-inspired system developed for supervised tabular datasets. The dashboard allows users to upload CSV datasets, inspect dataset quality, receive target-column recommendations, automatically detect classification or regression tasks, preprocess data, train multiple candidate models, compare their performance, recommend the best-performing model, generate predictions, and export results. The system was implemented using Python, Streamlit, Pandas, Scikit-learn, Plotly, Matplotlib, and Joblib. In the final experiment using a breast cancer diagnostic dataset containing 569 records and 32 attributes, the system selected *diagnosis* as the target column, detected a classification task, trained seven classification models, and selected Support Vector Machine (SVM) as the best-performing model with a Weighted F1 Score of 0.9735. The results show that the proposed dashboard can support beginner users in understanding the complete supervised machine learning workflow while reducing common errors related to target selection, preprocessing, model selection, and result interpretation.**

***Index Terms*—Automated Machine Learning, AutoML, Streamlit, Scikit-learn, Model Selection, Classification, Regression, Data Science Dashboard, Machine Learning Education, Support Vector Machine**

I. Introduction

Machine learning is widely used in healthcare, education, business, finance, engineering, public policy, and scientific research. It allows computers to learn patterns from data and make predictions or decisions without being explicitly programmed for every possible case. However, developing a reliable machine learning system is not limited to choosing one algorithm. A proper machine learning workflow includes several stages such as dataset understanding, target variable selection, preprocessing, feature handling, model training, model comparison, evaluation, interpretation, prediction, and reporting.

For beginner users, this complete workflow can be difficult to understand. In many student-level projects, a single algorithm is selected manually, trained on a dataset, and evaluated using one metric such as accuracy. This approach may be acceptable for basic demonstrations, but it has several weaknesses. First, it does not confirm whether the selected algorithm is the most suitable model for the dataset. Second, it may ignore important preprocessing steps such as handling missing values, encoding categorical features, and scaling numeric features. Third, users may accidentally select the wrong target column, such as an ID or serial number, which leads to meaningless results. Fourth, results may not be presented visually, making it difficult for users to understand what happened inside the machine learning process.

Automated Machine Learning, commonly known as AutoML, aims to reduce manual effort by automating parts of the machine learning pipeline. AutoML systems can automate model selection, preprocessing, hyperparameter tuning, and evaluation. However, many existing AutoML tools are either designed for expert users or focus mainly on performance optimization. They may provide strong predictive models, but they do not always explain the workflow clearly for beginners.

This paper presents *Smart AutoML Pro Dashboard*, a visual and interpretable AutoML-inspired system designed for supervised tabular machine learning tasks. The system is developed as a web-based dashboard where users can upload a CSV dataset, select a target column, view dataset quality, receive target recommendations, train multiple machine learning models, compare their performance, and export results. The system is designed not only to automate model comparison but also to make the process understandable.

The key idea behind this project is that a beginner user should not have to manually guess which model is best. Instead, the dashboard should test multiple models, compare their performance, and explain the result in a clear and visual way.

A. Motivation

The motivation behind this work comes from a common problem in beginner machine learning projects: students often build one fixed model without comparing it against alternatives. For example, a student may create a Random Forest

model for cancer diagnosis or a KNN model for student performance prediction without knowing whether that model is actually the best choice. A more systematic approach is needed.

The proposed dashboard addresses this by providing:

- a guided target selection process;
- automatic classification/regression detection;
- automatic preprocessing;
- training of multiple candidate models;
- visual comparison of results;
- explanation of the selected model;
- prediction interface;
- exportable report and trained model file.

B. Main Contributions

The main contributions of this project are:

1) Development of a visual AutoML-inspired dashboard for tabular supervised learning.
2) Design of a target advisor that recommends suitable prediction targets and discourages invalid columns such as IDs.
3) Integration of a data cleaning advisor for identifying missing values, ID-like columns, and other data quality issues.
4) Automatic detection of classification and regression tasks.
5) Implementation of automatic preprocessing for numeric and categorical variables.
6) Training and comparison of multiple machine learning models.
7) Visual result interpretation through model leaderboard, charts, confusion matrix, and prediction output.
8) Export of project report, model leaderboard, cleaning advice, and trained model file.

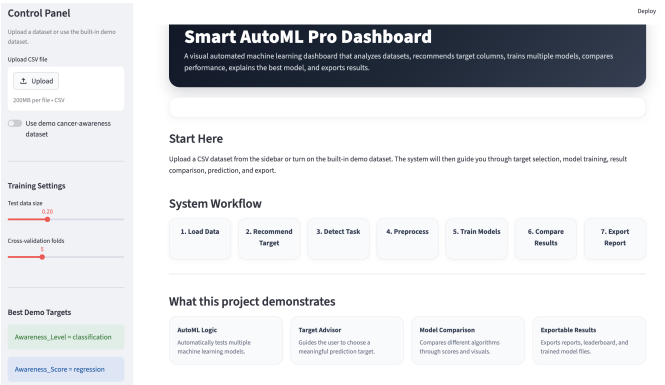


Fig. 1: Main interface of the Smart AutoML Pro Dashboard.

II. Background and Related Work

A. Machine Learning Workflow

A supervised machine learning workflow typically begins with a dataset containing input features and a target variable. The target variable is the outcome that the model should predict. If the target contains categories, the problem is known as classification. If the target contains continuous numeric values, the problem is known as regression.

A standard supervised machine learning workflow includes:

1) data collection;
2) data cleaning;
3) target variable selection;
4) feature preprocessing;
5) train-test split;
6) model training;
7) model evaluation;
8) model comparison;
9) prediction;
10) reporting or deployment.

Errors in any of these stages can affect model performance. For example, using an ID column as a feature may create misleading patterns, while selecting an ID column as the target makes the prediction task meaningless.

B. Automated Machine Learning

Automated Machine Learning aims to automate repetitive and complex parts of the machine learning workflow. It can include automatic preprocessing, algorithm selection, hyper-parameter optimization, and model evaluation. AutoML is useful because it reduces the amount of manual trial and error required in model development.

Several AutoML frameworks have been developed. Auto-sklearn uses Bayesian optimization, meta-learning, and ensemble construction. TPOT uses genetic programming to optimize machine learning pipelines. AutoGluon focuses on strong tabular prediction using automated ensembling. FLAML focuses on fast and lightweight AutoML under resource constraints.

Although these systems are powerful, they can be difficult for beginners to interpret. Many tools return a model or leaderboard but do not provide an educational, step-by-step explanation of target selection, preprocessing, and model comparison. The proposed dashboard is therefore designed as an educational and interpretable AutoML system rather than a production-grade AutoML optimizer.

C. Need for Visual Machine Learning Tools

Visual dashboards help users understand data and models more effectively. For beginner users, a dashboard interface can make machine learning less abstract by showing dataset summaries, charts, target distribution, class balance, model comparison, and evaluation visuals. This project uses Streamlit to create an interactive dashboard that presents the machine learning process in a structured and understandable form.

III. Problem Statement

Beginner machine learning users face several challenges when developing supervised learning models. These challenges include selecting the correct target column, understanding whether the problem is classification or regression, preprocessing different data types, selecting suitable models, evaluating performance, and interpreting results.

In many academic projects, the model development process is oversimplified. A single algorithm is selected manually, and the result is reported without comparing it against alternative algorithms. This can lead to incomplete analysis and weak justification for the selected model.

The problem addressed in this paper is:

> How can a beginner-friendly visual system automate key stages of supervised machine learning while making target selection, preprocessing, model comparison, prediction, and reporting easier to understand?

The proposed dashboard solves this problem by creating an end-to-end visual system that automates and explains the machine learning workflow.

IV. Objectives

The main objective of this project is to develop an interpretable visual AutoML dashboard for supervised tabular datasets.

The specific objectives are:

1) To allow users to upload CSV datasets.
2) To provide a built-in demo dataset for testing.
3) To display dataset overview, missing values, duplicates, and data types.
4) To calculate a dataset quality score.
5) To recommend suitable target columns.
6) To validate selected target columns.
7) To detect classification or regression tasks automatically.
8) To preprocess numeric and categorical features automatically.
9) To remove ID-like columns from model features.
10) To train multiple machine learning models.
11) To compare models using relevant evaluation metrics.
12) To recommend the best-performing model.
13) To visualize results through charts and tables.
14) To allow prediction on new user-entered cases.
15) To export reports, leaderboards, cleaning advice, and trained model files.

V. Proposed System

The proposed system is named *Smart AutoML Pro Dashboard*. It is an interactive web-based system designed to simplify machine learning model selection for tabular datasets.

A. System Overview

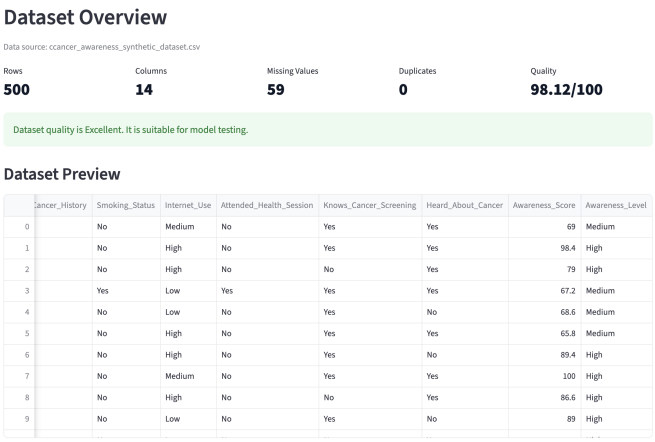
The dashboard follows a sequential workflow. First, the user uploads a dataset or uses the built-in demo dataset. Then, the system analyzes the dataset and provides a target advisor. After the user selects a target column, the system detects whether the task is classification or regression. Next, it applies preprocessing, trains candidate models, compares model performance, and recommends the best model. Finally, the user can make predictions and export results.

Fig. 2: Overall workflow of the proposed Smart AutoML Pro Dashboard.

B. System Modules

The system consists of the following six major modules:

1) **Overview Module**: Shows dataset summary, quality score, column summary, missing values, and data type distribution.
2) **Advisor Module**: Provides target recommendations, target validation, data cleaning advice, class balance, and target distribution.
3) **Training Module**: Displays preprocessing steps, candidate models, and starts automated model training.
4) **Results Module**: Shows the best model, model leaderboard, evaluation chart, confusion matrix or regression plot, and model insight.
5) **Prediction Module**: Allows the user to enter values for a new case and generate predictions.
6) **Export Module**: Provides downloadable reports, leaderboard, cleaning advice, and best trained model file.



Dataset Overview

Data source: ccancer_awareness_synthetic_dataset.csv

Rows	Columns	Missing Values	Duplicates	Quality
500	14	59	0	98.12/100

Dataset quality is Excellent. It is suitable for model testing.

Dataset Preview

	ancer_History	Smoking_Status	Internet_Use	Attended_Health_Session	Knows_Cancer_Screening	Heard_About_Cancer	Awareness_Score	Awareness_Level
0		No	Medium	No	Yes	Yes	69	Medium
1		No	High	No	Yes	Yes	98.4	High
2		No	High	No	No	Yes	79	High
3		Yes	Low	Yes	Yes	Yes	67.2	Medium
4		No	Low	No	Yes	No	68.6	Medium
5		No	High	No	Yes	Yes	65.8	Medium
6		No	High	No	Yes	No	89.4	High
7		No	Medium	No	Yes	Yes	100	High
8		No	High	No	No	Yes	86.6	High
9		No	Low	No	Yes	No	89	High

Fig. 3: Overview module showing dataset summary and data quality information.

VI. System Architecture

The system architecture is divided into five layers:

1) Input Layer
2) Data Analysis Layer
3) Preprocessing Layer
4) Model Training and Evaluation Layer
5) Output and Export Layer

A. Input Layer

The input layer accepts CSV datasets uploaded by the user. It also provides a built-in synthetic cancer-awareness dataset for testing. The uploaded dataset is read using Pandas and passed to the analysis layer.

B. Data Analysis Layer

The data analysis layer inspects the dataset and calculates:

- number of rows;
- number of columns;
- missing values;
- duplicate rows;
- unique values per column;
- data types;
- ID-like columns;
- dataset quality score.

C. Preprocessing Layer

The preprocessing layer prepares the dataset for model training. Numeric features are processed using median imputation and standard scaling. Categorical features are processed using most frequent value imputation and one-hot encoding.

D. Model Training and Evaluation Layer

This layer trains multiple candidate models depending on whether the task is classification or regression. Each model is evaluated using relevant metrics. The best model is selected based on the main evaluation metric.

E. Output and Export Layer

The output layer displays the result dashboard and allows the user to export the project report, model leaderboard, data cleaning advice, and trained model file.

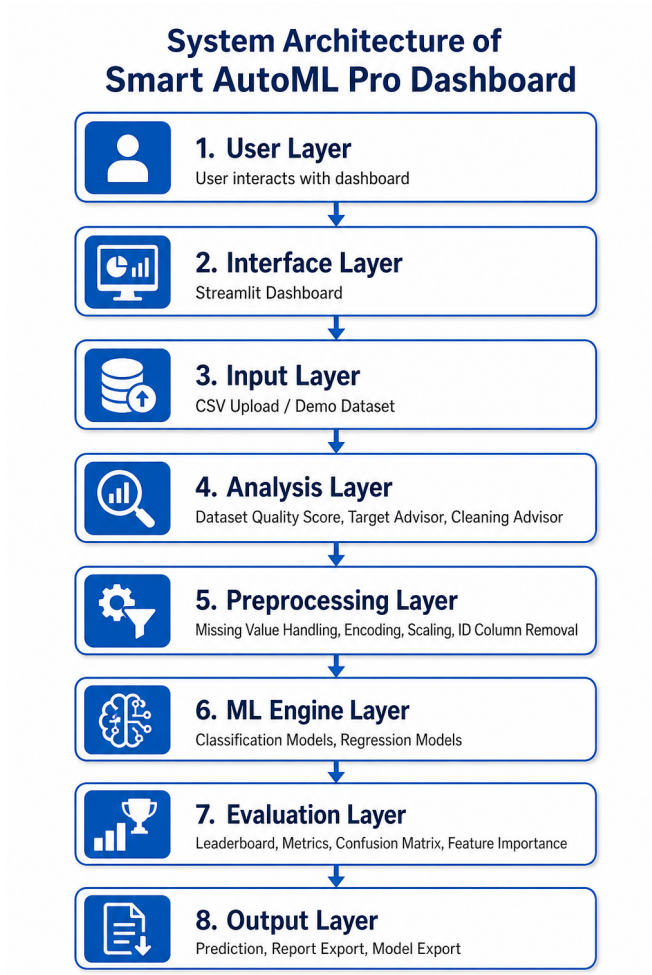


Fig. 4: Layered architecture of the proposed system.

VII. Methodology

A. Dataset Loading

The system accepts tabular datasets in CSV format. When a CSV file is uploaded, the system attempts to read it using Pandas. If the file is unreadable, an error message is displayed. If the file is empty, the system stops execution and asks the user to provide a valid dataset.

The system also removes unnamed index columns that are commonly created when dataframes are exported to CSV files.

B. Dataset Quality Scoring

A dataset quality score is calculated to give users a quick view of dataset readiness. The score is based on missing values, duplicate rows, and ID-like columns.

Let:

$$Q = 100 - P_m - P_d - P_i$$

where Q is the dataset quality score, P_m is the penalty for missing values, P_d is the penalty for duplicate rows, and P_i is the penalty for ID-like columns.

The quality score is categorized as:

- Excellent: score greater than or equal to 90.
- Good: score between 75 and 89.
- Moderate: score between 55 and 74.
- Needs Cleaning: score below 55.

C. Target Recommendation

Target selection is one of the most important steps in supervised learning. The system recommends target columns based on column name, data type, number of unique values, missing values, and whether the column appears to be an identifier.

Columns with names such as *diagnosis*, *outcome*, *target*, *class*, *level*, *status*, and *score* receive higher target scores. Columns such as *id*, *student_id*, *record_id*, and serial numbers are discouraged.

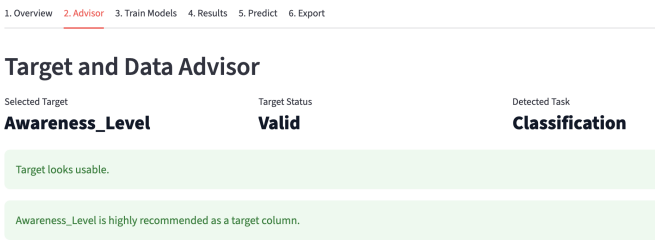


Fig. 5: Target advisor module showing recommended and weak target columns.

D. Target Validation

After the user selects a target column, the system validates whether it is suitable. A target column is considered unsuitable if:

- it appears to be an ID or unique identifier;
- it has fewer than two unique values;
- it has excessive missing values;
- it does not represent a meaningful prediction outcome.

E. Task Type Detection

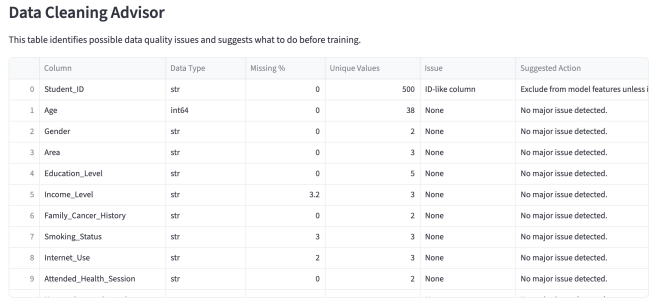
The system automatically detects whether the problem is classification or regression. If the target column is categorical or contains a small number of unique values, the task is classified as a classification problem. If the target column contains continuous numeric values, the task is classified as a regression problem.

F. Data Cleaning Advisor

The data cleaning advisor identifies possible issues in each column. It checks:

- missing value percentage;
- unique value count;
- ID-like columns;
- constant columns;
- high-cardinality categorical columns.

For each issue, the system provides a suggested action.



Data Cleaning Advisor

This table identifies possible data quality issues and suggests what to do before training.

	Column	Data Type	Missing %	Unique Values	Issue	Suggested Action
0	Student_ID	str	0	500	ID-like column	Exclude from model features unless i
1	Age	int64	0	38	None	No major issue detected.
2	Gender	str	0	2	None	No major issue detected.
3	Area	str	0	3	None	No major issue detected.
4	Education_Level	str	0	5	None	No major issue detected.
5	Income_Level	str	3.2	3	None	No major issue detected.
6	Family_Cancer_History	str	0	2	None	No major issue detected.
7	Smoking_Status	str	3	3	None	No major issue detected.
8	Internet_Use	str	2	3	None	No major issue detected.
9	Attended_Health_Session	str	0	2	None	No major issue detected.

Fig. 6: Data cleaning advisor showing detected issues and suggested actions.

G. Preprocessing Pipeline

The preprocessing pipeline is implemented using Scikit-learn's *Pipeline* and *ColumnTransformer*. Numeric and categorical features are processed separately.

1) Numeric Features: Numeric features are processed using:

- median imputation;
- standard scaling.

2) Categorical Features: Categorical features are processed using:

- most frequent value imputation;
- one-hot encoding.

H. Train-Test Split

The dataset is divided into training and testing sets. The default setting uses 80% of the data for training and 20% for testing. For classification tasks, stratified splitting is applied where possible to preserve target class distribution.

I. Model Training

The system trains different models based on the detected task type.

For classification tasks, the system trains:

- Logistic Regression;
- K-Nearest Neighbors Classifier;
- Decision Tree Classifier;
- Random Forest Classifier;
- Support Vector Machine;
- Naive Bayes;
- Gradient Boosting Classifier.

For regression tasks, the system trains:

- Linear Regression;
- Ridge Regression;
- Polynomial Regression;
- K-Nearest Neighbors Regressor;
- Decision Tree Regressor;
- Random Forest Regressor;
- Support Vector Regressor;
- Gradient Boosting Regressor.

J. Evaluation Metrics

For classification, the system calculates accuracy, precision, recall, and Weighted F1 Score. Weighted F1 Score is used as the main selection metric because it considers both precision and recall while accounting for class distribution.

For regression, the system calculates Mean Absolute Error, Root Mean Squared Error, and R-squared Score. R-squared Score is used as the main selection metric.

VIII. Implementation

A. Development Environment

The system was developed in Python. Streamlit was used to create the web interface. Pandas and NumPy were used for data handling. Scikit-learn was used for preprocessing, model training, and evaluation. Plotly and Matplotlib were used for visualization. Joblib was used for saving the trained model.

TABLE I: Software Tools and Libraries Used

Library	Purpose
Python	Main programming language
Streamlit	Interactive web dashboard
Pandas	Data loading and manipulation
NumPy	Numerical operations
Scikit-learn	Preprocessing, model training, and evaluation
Plotly	Interactive charts and visualizations
Matplotlib	Confusion matrix and static visualizations
Joblib	Trained model export

B. Dashboard Layout

The dashboard is organized into six tabs:

1) Overview
2) Advisor
3) Train Models
4) Results
5) Predict
6) Export

This structure allows the user to move through the project workflow step by step.

C. Target Advisor Implementation

The target advisor assigns a score to each column. Columns with meaningful target-like names receive higher scores. ID-like columns receive lower scores and are marked as unsuitable. This helps prevent common mistakes such as selecting *id* or *student_id* as the output variable.

D. Training Module Implementation

The training module creates a pipeline for each model. Each pipeline includes preprocessing and the machine learning algorithm. This ensures that every model is trained using the same preprocessing logic, making the comparison fair.

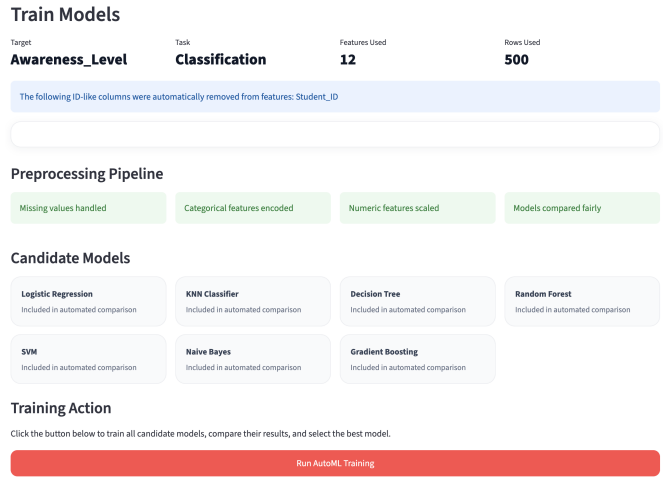


Fig. 7: Training module showing candidate models and preprocessing steps.

E. Export Module Implementation

The export module allows users to download:

- project report in text format;
- model leaderboard in CSV format;
- data cleaning advice in CSV format;
- best trained model in PKL format.

IX. Experimental Setup

A. Dataset Description

The final experiment was performed using a breast cancer diagnostic dataset. The dataset contains 569 records and 32 columns. The target variable selected was *diagnosis*, which represents the diagnostic category. The dataset contained no missing values and no duplicate rows. The dashboard identified *id* as an ID-like column and excluded it from the model features.

TABLE II: Dataset Summary

Attribute	Value
Rows	569
Columns	32
Target Column	diagnosis
Missing Values	0
Duplicate Rows	0
Dataset Quality Score	98.5/100
Detected Task	Classification
Excluded ID-like Column	id

B. Experimental Procedure

The experimental procedure was as follows:

1) The breast cancer dataset was uploaded to the dashboard.
2) The dashboard analyzed the dataset structure.
3) The target advisor recommended target columns.
4) The target column *diagnosis* was selected.
5) The system detected the task as classification.
6) The ID-like column *id* was removed from model features.
7) Numeric features were scaled.
8) Seven classification models were trained.

9) Models were compared using Weighted F1 Score.
10) The best model was selected and exported.

X. Results

A. Model Training Result

The dashboard successfully trained all seven classification models. The models tested were:

- Support Vector Machine;
- Logistic Regression;
- Random Forest;
- Gradient Boosting;
- K-Nearest Neighbors Classifier;
- Decision Tree;
- Naive Bayes.

B. Best Model Result

The best-performing model was Support Vector Machine. It achieved a Weighted F1 Score of 0.9735.

TABLE III: Best Model Result

Metric	Result
Best Model	Support Vector Machine
Task Type	Classification
Main Metric	Weighted F1 Score
Best Score	0.9735
Successful Models	7 out of 7

Model Leaderboard

	Model	Accuracy	Precision	Recall	F1 Score	CV Score	Main Score	Status	Error
0	Decision Tree	1	1	1	1	1	1	Success	
1	Gradient Boosting	1	1	1	1	1	1	Success	
2	Random Forest	0.97	0.96	0.97	0.9648	0.9789	0.9648	Success	
3	SVM	0.94	0.9304	0.94	0.9352	0.9276	0.9352	Success	
4	Logistic Regression	0.91	0.9064	0.91	0.9073	0.959	0.9073	Success	
5	KNN Classifier	0.91	0.8982	0.91	0.9027	0.8742	0.9027	Success	
6	Naive Bayes	0.8	0.8649	0.8	0.8271	0.7815	0.8271	Success	

Fig. 8: Model leaderboard showing performance comparison of trained classifiers.

C. Confusion Matrix

For classification tasks, the dashboard displays a confusion matrix. The confusion matrix helps show how many predictions were correct and how many were incorrect for each class.

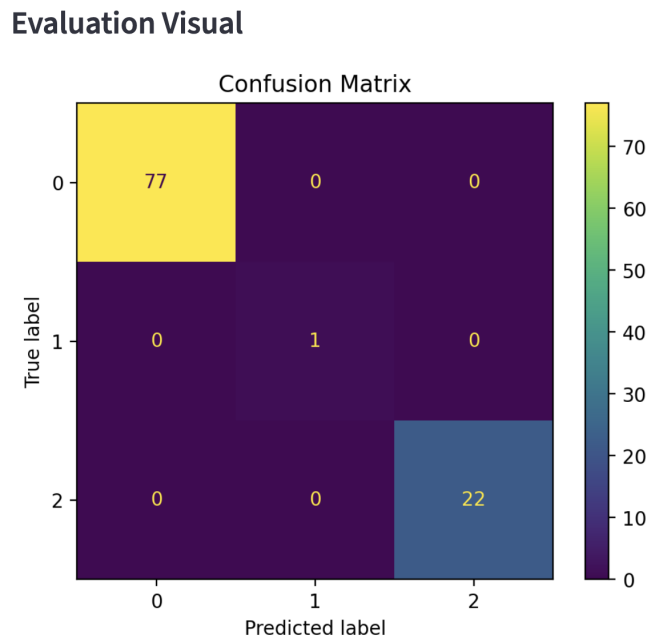


Fig. 9: Confusion matrix generated for the best classification model.

D. Interpretation of Results

The Weighted F1 Score of 0.9735 indicates strong classification performance. This means the selected model performed well across the diagnostic classes. SVM was selected because it achieved the highest score among all trained models. SVM is effective for structured datasets because it attempts to find a decision boundary that separates classes.

The dashboard also reported that the top models performed closely. This means that, although SVM achieved the best score, other models may also be competitive. In practical machine learning, the final model selection should consider not only accuracy or F1 score but also interpretability, stability, computational cost, and domain requirements.

E. Prediction Output

The dashboard includes a prediction module where the user can enter a new case and generate a prediction using the best selected model.

TABLE IV: Example prediction generated by the Smart AutoML system

Feature	Value
Age	30
Gender	Female
Area	Urban
Education Level	Graduate
Income Level	Low
Family Cancer History	Yes
Smoking Status	No
Internet Use	High
Attended Health Session	No
Knows Cancer Screening	No
Heard About Cancer	Yes
Awareness Score	83.85
Predicted Output	**High**

XI. Discussion

The results show that the dashboard successfully performs an end-to-end supervised machine learning workflow. It begins with dataset loading and continues through target recommendation, task detection, preprocessing, model training, evaluation, prediction, and export.

The system is particularly useful for educational purposes. It helps beginners understand that machine learning model development is not simply about choosing one algorithm. Instead, the process requires correct target selection, appropriate preprocessing, model comparison, and meaningful evaluation.

The dashboard also improves transparency. The user can see which models were trained, how they performed, and why one model was selected. This visual and explanatory approach makes the system more understandable than a code-only machine learning script.

XII. Strengths of the System

The main strengths of the proposed dashboard are:

- It provides a user-friendly visual interface.
- It supports both classification and regression.
- It recommends suitable target columns.
- It detects invalid ID-like target columns.
- It provides data cleaning advice.
- It automatically preprocesses numeric and categorical features.
- It trains multiple models instead of relying on one manually selected algorithm.
- It compares models visually.
- It provides prediction functionality.
- It exports reports, leaderboards, cleaning advice, and trained models.

XIII. Limitations

Although the dashboard is useful, it has some limitations:

- It currently supports only tabular CSV datasets.
- It does not support image, audio, video, or natural language data.
- It uses limited default hyperparameters.
- It does not yet include advanced hyperparameter tuning.
- It does not include SHAP or LIME explainability.
- It does not perform advanced feature selection.
- It does not include fairness or bias analysis.
- It is designed mainly for learning and demonstration rather than production deployment.

XIV. Future Work

Future versions of the dashboard can include:

- hyperparameter tuning using GridSearchCV or RandomizedSearchCV;
- SHAP-based model explainability;
- LIME explanations for individual predictions;
- PDF report generation;
- feature selection methods;
- clustering and unsupervised learning;
- time-series forecasting;
- database integration;
- REST API deployment;
- user login and saved project history;
- fairness and bias detection;
- comparison with established AutoML frameworks;
- automatic data leakage detection.

XV. Conclusion

This paper presented Smart AutoML Pro Dashboard, an interpretable visual AutoML-inspired system for supervised tabular machine learning tasks. The system allows users to upload datasets, receive target recommendations, detect classification or regression tasks, preprocess data, train multiple models, compare performance, make predictions, and export results.

The final experiment using a breast cancer diagnostic dataset showed that the dashboard successfully selected *diagnosis* as the target variable, detected the task as classification, trained seven classification models, and selected Support Vector Machine as the best-performing model with a Weighted F1 Score of 0.9735.

The project demonstrates that AutoML concepts can be presented in a simplified, visual, and educational way. While the system does not replace advanced industrial AutoML frameworks, it provides a useful dashboard for understanding supervised machine learning workflows and avoiding common beginner mistakes.

References

[1] M. Feurer, A. Klein, K. Eggensperger, J. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[2] M. Feurer, A. Klein, K. Eggensperger, J. Springenberg, M. Blum, and F. Hutter, "Auto-sklearn: Efficient and robust automated machine learning," in *Automated Machine Learning: Methods, Systems, Challenges*, Springer, 2019, pp. 113–134.

[3] R. S. Olson and J. H. Moore, "TPOT: A tree-based pipeline optimization tool for automating machine learning," in *Proceedings of the Workshop on Automatic Machine Learning*, PMLR, vol. 64, pp. 66–74, 2016.

[4] N. Erickson, J. Mueller, A. Shirkov, H. Zhang, P. Larroy, M. Li, and A. Smola, "AutoGluon-Tabular: Robust and accurate AutoML for structured data," in *ICML Workshop on Automated Machine Learning*, 2020.

[5] C. Wang, Q. Wu, M. Weimer, and E. Zhu, "FLAML: A fast and lightweight AutoML library," in *Proceedings of Machine Learning and Systems*, vol. 3, pp. 434–447, 2021.

[6] F. Hutter, L. Kotthoff, and J. Vanschoren, *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019.

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[8] W. McKinney, "Data structures for statistical computing in Python," in *Proceedings of the 9th Python in Science Conference*, pp. 56–61, 2010.

[9] A. Treuille, T. Teixeira, and A. Kelly, "Streamlit: The fastest way to build data apps," Streamlit Inc., 2019. [Online]. Available: https://streamlit.io

[10] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[12] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[13] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[14] D. J. Hand and K. Yu, "Idiot's Bayes—not so stupid after all?," *International Statistical Review*, vol. 69, no. 3, pp. 385–398, 2001.

[15] D. Dua and C. Graff, "UCI Machine Learning Repository," University of California, Irvine, School of Information and Computer Sciences, 2019. [Online]. Available: https://archive.ics.uci.edu/

[16] W. H. Wolberg, W. N. Street, and O. L. Mangasarian, "Breast cancer Wisconsin diagnostic dataset," University of Wisconsin Hospitals, Madison, 1995.

[17] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.

[18] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you? Explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, 2016.

[19] S. M. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[20] R. Wirth and J. Hipp, "CRISP-DM: Towards a standard process model for data mining," in *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 2000.